

URL Shortener - Documento técnico de entrega

1. Resumen

El proyecto implementa una aplicación de URLs cortas con React, Node.js y TypeScript. Permite:

- crear una URL corta con código generado o alias personalizado;
- resolver el código mediante un redirect `302`;
- cachear resoluciones en Redis;
- publicar un evento por cada acceso válido;
- procesar esos eventos asincrónicamente con un worker;
- persistir clicks de forma idempotente;
- consultar estadísticas básicas por código.

La solución utiliza React, Vite, Tailwind CSS, Express, MongoDB, Redis, RabbitMQ y Docker Compose. La API y el worker son procesos separados que comparten modelos, configuración y contratos.

2. Arquitectura

```

Cliente
  |
  v
Frontend React
  |
  v
API Express
  |---- MongoDB: URLs y estadísticas
  |---- Redis: cache de resolución
  `---- RabbitMQ: publicación de accesos
        |
        v
      Worker
        |
        v
MongoDB: click_events
```

El código se organiza por funcionalidad:

```
frontend/  
src/  
  api/  
  worker/  
  infrastructure/  
    config/  
    mongo/  
    redis/  
    rabbitmq/  
modules/  
  urls/  
  clicks/  
shared/  
  errors/  
  logger/
```

Para los módulos HTTP se respeta la dirección:

```
route -> controller -> service -> repository -> model -> MongoDB
```

- Las rutas declaran método y path.
- Los controllers traducen HTTP y permanecen delgados.
- Los services validan entradas e implementan casos de uso.
- Los repositories encapsulan MongoDB y traducen errores del proveedor.
- Los models definen documentos e índices.
- Infraestructura administra conexiones externas y lifecycle.

3. Procesos

API

La API conecta primero a MongoDB, dependencia obligatoria. Redis y RabbitMQ son degradables:

- si Redis falla, la resolución consulta MongoDB;
- si RabbitMQ falla, el redirect continúa sin registrar la métrica.

Al recibir **SIGINT** o **SIGTERM**, deja de aceptar requests y cierra RabbitMQ, Redis y MongoDB.

Worker

El worker conecta a MongoDB y RabbitMQ, crea un canal consumidor y procesa la cola

tinyurl.accessed.persist.

Usa acknowledgements manuales:

- evento persistido: **ack**;
- **eventId** duplicado: **ack**;
- mensaje inválido: **reject** sin requeue;
- error de persistencia: **nack** con requeue.

`prefetch(10)` limita a diez los mensajes entregados al proceso que todavía no recibieron `ack` o `nack`.

4. Modelos

ShortUrl

Colección Mongoose: `shorturls`.

```
type ShortUrl = {
  code: string;
  originalUrl: string;
  createdAt: Date;
  updatedAt: Date;
};
```

`code` tiene índice único. Contiene tanto códigos generados como alias.

ClickEvent

Colección explícita: `click_events`.

```
type ClickEvent = {
  eventId: string;
  code: string;
  occurredAt: Date;
  ip?: string;
  userAgent?: string;
  createdAt: Date;
  updatedAt: Date;
};
```

Índices:

- `eventId` único para idempotencia;
- `{ code: 1, occurredAt: -1 }` para buscar el último acceso.

La API espera la inicialización de ambos modelos antes de abrir el puerto HTTP y el worker espera la inicialización de `ClickEventModel` antes de consumir. Esto garantiza que los índices críticos estén disponibles antes de procesar tráfico.

Cada acceso se guarda como un evento individual. Para este assessment, las estadísticas cuentan esos documentos. En un sistema de alto volumen, el worker mantendría además un agregado `click_stats` por código, actualizado con `$inc` y `$max`.

5. Módulo de URLs

Schemas

`url.schemas.ts` valida:

- `originalUrl` como URL HTTP o HTTPS;

- alias opcional;
- rechazo del alias reservado `health`;
- códigos de 3 a 50 caracteres;
- letras, números, - y _.

Repository

`url.repository.ts` contiene:

- `createShortUrl`: crea una URL y traduce duplicate key `11000`;
- `findShortUrlByCode`: busca una URL por código;
- `shortUrlCodeExists`: verifica existencia para alias.

MongoDB es la autoridad final sobre unicidad. Redis nunca decide si un alias está disponible.

Cache

`url.cache.ts` encapsula Redis:

```
short-url:{code} -> originalUrl
```

El TTL se configura con `REDIS_CACHE_TTL_SECONDS`.

Service

`url.service.ts` implementa:

- creación con alias;
- generación mediante `randomBytes(6).toString("base64url")`;
- hasta cinco reintentos ante colisiones generadas;
- resolución cache-aside;
- construcción y publicación del evento de acceso.

Los fallos de lectura o escritura de Redis se registran y no interrumpen la resolución. Los fallos de publicación tampoco bloquean el redirect.

Controllers y rutas

```
POST /api/urls
GET /:code
```

Los controllers solo pasan datos al service y serializan el caso exitoso.

6. Módulo de clicks

Contrato del evento

```
type TinyUrlAccessedEvent = {
  eventId: string;
  type: "tinyurl.accessed.v1";
  occurredAt: string;
  data: {
    code: string;
    ip?: string;
    userAgent?: string;
  };
};
```

El contrato se valida con Zod antes de publicar y al consumir.

Publisher

`click.publisher.ts`:

- serializa el evento como JSON;
- publica en `tinyurl.events`;
- usa routing key `tinyurl.accessed.v1`;
- marca el mensaje como persistente;
- espera confirmación del broker mediante un confirm channel;
- limita esa espera con `RABBITMQ_PUBLISH_TIMEOUT_MS`.

Repository

`click.repository.ts` contiene:

- `saveClickEvent`: persiste el evento;
- `countClicksByCode`: cuenta eventos por código;
- `findLatestClickByCode`: obtiene el acceso más reciente.

Si MongoDB rechaza un `eventId` repetido con error `11000`, devuelve `duplicate`. El consumer lo considera exitoso y envía `ack`. La idempotencia depende del índice único, no de una consulta previa vulnerable a condiciones de carrera.

Consumer

`click.consumer.ts`:

1. convierte el buffer a JSON;
2. valida el evento;
3. llama al repository;
4. decide `ack`, `reject` o `nack` con requeue.

Los eventos procesados correctamente se registran en nivel `debug`; errores de persistencia se registran en `error`.

Estadísticas

`click.service.ts`:

1. valida el código;
2. verifica que la URL exista;
3. consulta en paralelo el total y el último click;
4. devuelve:

```
{
  "code": "example",
  "totalClicks": 12,
  "lastClick": "2026-06-14T19:00:00.000Z"
}
```

La ruta es:

```
GET /api/stats/:code
```

Está registrada antes de `GET /:code` para que Express no interprete `api` como un código corto.

7. Redis

La resolución usa cache-aside:

1. buscar en Redis;
2. ante hit, devolver la URL;
3. ante miss o error, buscar en MongoDB;
4. guardar el resultado en Redis;
5. continuar con el redirect.

No se precarga Redis al crear una URL y no se implementa negative caching. MongoDB sigue siendo la fuente de verdad.

El prefijo `short-url`: funciona como namespace y evita colisiones con futuras claves, por ejemplo `click-stats:{code}`.

8. RabbitMQ

Topología:

```
exchange durable: tinyurl.events
tipo:             direct
routing key:      tinyurl.accessed.v1
cola durable:     tinyurl.accessed.persist
```

`assertExchange`, `assertQueue` y `bindQueue` se ejecutan al conectar para garantizar que la topología exista.

La publicación del evento y el redirect no forman una transacción distribuida. Si RabbitMQ falla, puede perderse una métrica aunque el redirect sea correcto. Una garantía más fuerte requeriría un transactional outbox.

9. Manejo de errores

Todas las respuestas públicas usan:

```
{
  "error": {
    "code": "MACHINE_READABLE_CODE",
    "message": "Human-readable message",
    "details": []
  }
}
```

`details` es opcional.

AppError

Representa errores esperados con código público y status HTTP:

- `SHORT_URL_CODE_CONFLICT` -> 409;
- `SHORT_URL_CODE_GENERATION_FAILED` -> 500;
- `SHORT_URL_NOT_FOUND` -> 404.

ZodError

Los services validan entradas runtime. El middleware transforma errores Zod en:

```
400 VALIDATION_ERROR
```

errorHandler

Se registra después de todas las rutas. Express 5 propaga automáticamente promesas rechazadas:

```
controller/service throws
-> Express
-> errorHandler
-> respuesta HTTP
```

Los controllers no duplican `try/catch`. Los errores desconocidos se registran y se responden como `500 INTERNAL_ERROR`, sin stack ni detalles internos.

El middleware también traduce JSON malformado a `400 INVALID_JSON`, payloads mayores al límite de Express a `413 PAYLOAD_TOO_LARGE` y rutas no registradas a `404 ROUTE_NOT_FOUND`.

10. Logging

Pino genera logs estructurados a stdout. `pino-http`:

- asigna o propaga `x-request-id`;
- registra método, path, status y duración;
- usa `info` para `2xx/3xx`, `warn` para `4xx` y `error` para `5xx`;
- excluye `/health` para reducir ruido.

Authorization, cookies y el header de redirect `Location` se redactan. Las URLs originales no se registran porque sus query params podrían contener información sensible.

Los módulos usan child loggers con un campo `module` estable.

11. Librerías principales

Librería	Uso
<code>express</code>	Servidor HTTP, rutas y middleware
<code>mongoose</code>	Modelos, índices y consultas MongoDB
<code>redis</code>	Cliente Redis y cache de resolución
<code>amqplib</code>	Conexión, publicación y consumo RabbitMQ
<code>zod</code>	Validación runtime y tipos inferidos
<code>pino</code>	Logging estructurado
<code>pino-http</code>	Logs HTTP y request IDs
<code>dotenv</code>	Carga de <code>.env</code>
<code>vitest</code>	Tests unitarios
<code>supertest</code>	Tests HTTP
<code>tsx</code>	Ejecución TypeScript en desarrollo
<code>typescript-eslint</code>	Linting TypeScript
<code>cross-env</code>	Entorno de build portable entre sistemas operativos
<code>react</code>	Interfaz para operar la aplicación

Librería	Uso
<code>vite</code>	Servidor y build del frontend
<code>tailwindcss</code>	Estilos utilitarios
<code>@testing-library/react</code>	Tests de interacción del frontend

12. Endpoints

Crear URL

POST /api/urls

Body:

```
{
  "originalUrl": "https://example.com",
  "alias": "example"
}
```

Respuesta: 201 Created.

Resolver URL

GET /example

Respuesta: 302 Found con header Location.

Consultar estadísticas

GET /api/stats/example

Respuesta: 200 OK.

Salud

GET /health

Respuesta:

```
{ "status": "ok" }
```

Es un healthcheck superficial del proceso HTTP, no un diagnóstico profundo de dependencias.

13. Configuración y ejecución

Preparación:

```
cp .env.example .env
npm install
npm run docker:up
```

Servicios locales:

```
Frontend:          http://localhost:5173
API:               http://localhost:3000
MongoDB:          localhost:27017
Redis:            localhost:6379
RabbitMQ AMQP:     localhost:5672
RabbitMQ Management: http://localhost:15672
```

Variables principales:

Variable	Propósito
NODE_ENV	Entorno de ejecución
PORT	Puerto HTTP
FRONTEND_PORT	Puerto local del servidor Vite
APP_BASE_URL	Base usada para construir URLs cortas
VITE_API_ORIGIN	Origen público usado para abrir códigos
VITE_API_PROXY_TARGET	Destino del proxy API de Vite
MONGODB_URI	Conexión a MongoDB
REDIS_URL	Conexión a Redis
REDIS_CACHE_TTL_SECONDS	TTL de resoluciones cacheadas
RABBITMQ_URL	Conexión AMQP
RABBITMQ_EXCHANGE	Exchange de eventos
RABBITMQ_ACCESS_QUEUE	Cola consumida por el worker
RABBITMQ_PUBLISH_TIMEOUT_MS	Espera máxima de confirmación al publicar
LOG_LEVEL	Nivel mínimo de Pino

La configuración del backend se carga con dotenv y se valida con Zod al iniciar. Las variables FRONTEND_PORT, VITE_API_ORIGIN y VITE_API_PROXY_TARGET pertenecen a Compose/Vite y no al schema del backend. .env.example documenta los valores locales y .env no se versiona.

Comandos principales:

```
npm run typecheck
npm run lint
npm test
npm run build
npm run docker:ps
npm run docker:logs:api
npm run docker:logs:worker
```

14. Pruebas

La suite contiene 17 archivos y 66 tests. Cubre:

- schemas de URL y eventos;
- creación, colisiones y resolución;
- cache hit, miss y fallos Redis;
- publisher y confirmación RabbitMQ;
- timeout de confirmación RabbitMQ;
- JSON malformado, payload excedido y rutas inexistentes;
- idempotencia por `eventId`;
- políticas `ack`, `reject` y requeue;
- repository, service, controller y ruta de estadísticas;
- serialización global de errores.
- creación, apertura por código y estadísticas desde React.

También se verificaron manualmente contra Docker:

- resolución y población de Redis;
- publicación y consumo RabbitMQ;
- persistencia en `click_events`;
- dos accesos concurrentes;
- estadísticas antes y después de los accesos.

15. Decisiones y limitaciones

- MongoDB es la fuente de verdad.
- Redis es una optimización y nunca decide unicidad.
- RabbitMQ no bloquea redirects válidos.
- El consumidor es idempotente mediante índice único en `eventId`.
- Se usa `302` para que los accesos futuros sigan llegando a la aplicación.
- Las estadísticas cuentan eventos individuales por simplicidad.
- No se implementan usuarios, autenticación, edición ni analytics avanzados.
- No hay garantía transaccional entre redirect y publicación.

- No hay dead-letter queue ni reconexión RabbitMQ avanzada.
- La validación Zod asegura el contrato, no la autenticidad criptográfica del productor.

Para una evolución productiva se considerarían:

- agregado `click_stats` por código;
- transactional outbox;
- dead-letter queue;
- permisos mínimos y TLS para RabbitMQ;
- firma HMAC si existen productores internos no confiables;
- healthchecks profundos y observabilidad con métricas.